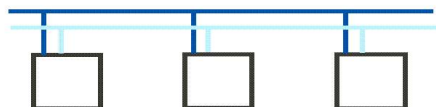


CAN Protocol

Full Guide



CANBus



CAN Introduction

CAN (Controller Area Network) is a robust serial communication protocol developed by Robert Bosch GmbH in 1983-1986 for automotive applications. It has become the standard for in-vehicle networking and industrial automation.

CAN VERSIONS

- CAN 2.0A: Standard 11-bit identifier
- CAN 2.0B: Extended 29-bit identifier
- CAN FD: Up to 64Bytes payload
- CAN XL: Up to 2048Bytes payload

NOTES & TIPS

The Guide's Focuses on
CAN2.0A, CAN2.0B
And CAN FD.

CAN ADVANTAGES

- Reduced complex point-to-point wiring (80%+ reduction)
- Eliminated heavy wiring harnesses in vehicles
- Enabled multi-master communication
- Provided deterministic real-time messaging
- Built-in error detection and fault tolerance

CAN EVOLUTION

Year	Milestone
1983	First Development (Bosch)
1986	First Spec Released
1991	CAN 2.0A/B specification
1993	ISO 11898 standardization
2003	ISO 11898 revised
2012	CAN FD introduced
2019	CAN XL announced

CAN TIMELINE



CAN Architecture

CAN PROTOCOL OSI MODEL

Layer 1: Physical Layer

- Electrical signaling: Voltage levels, bit timing, cable specifications
- Medium: Transmits raw bits as differential signals on CAN_H/CAN_L wires

Layer 2: Data Link Layer

- Framing: Structures data into CAN frames (SOF, ID, control, data, CRC, ACK, EOF)
- Error detection: CRC, bit monitoring, stuffing checks, ACK verification, fault confinement

Layer 3: Network Layer

- Addressing: Node IDs (1-127) and function codes define message routing
- Segmentation: Service Data Object transfers for data >8 bytes

Layer 4: Transport Layer

- Flow control: Block transfer protocol for efficient multi-frame data transfer
- Reliability: Acknowledgment and retry mechanisms for guaranteed delivery

Layer 5: Session Layer

- Session control: Heartbeat/node guarding monitors node presence
- Connection establishment: Boot-up message announces node initialization

Layer 6: Presentation Layer

- Data encoding: Object dictionary defines data types (integers, strings, floats)
- Data translation: EDS (Electronic Data Sheet) standardizes device description

Layer 7: Application Layer

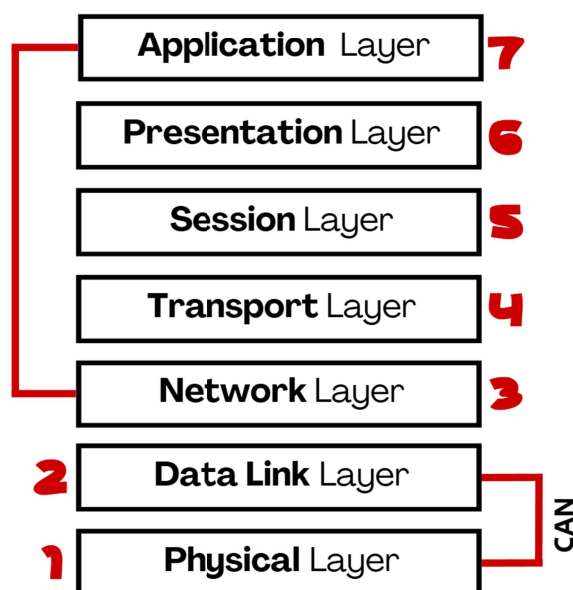
- Device profiles: Standardized behavior for device types (I/O, motion, drives)
- PDO (Process Data Objects): Real-time cyclic data exchange (sensors, actuators)

NOTES & TIPS

Other High-Level COMM:

- DeviceNet
- J1939
- SAE J2284

CANopen



CORE CHARACTERISTICS

Multi-Master:

- Any node can transmit when bus is idle
- No central master controller
- Distributed control system

Message-Based:

- Content-addressed (not destination-addressed)
- Messages identified by ID, not node address
- All nodes receive all messages

Priority System:

- Lower ID number = higher priority
- Deterministic arbitration
- No message loss during arbitration

NOTES & TIPS

Nodes ID Compared

To Select Dominant.

Rest of nodes are Receivers

Broadcast With Multi-cast:

- All nodes receive messages
- Each node filters based on acceptance criteria
- Multiple nodes can act on same message

Requirements:

- Linear bus topology
- Two-wire differential signaling (CAN_H, CAN_L)
- 120ohm termination resistors at both ends
- Twisted pair cable

Bus Length (max)

Bit Rate	Max Length
8 Mbps	16 m
1 Mbps	40m
500kbps	100 m
250kbps	250 m
125kbps	500 m
50kbps	1000 m

Message Frame Formats

FRAME TYPES

Frame Type	Purpose
Data Frame	Transmit data (0-8 bytes)
Remote Frame	Request data transmission
Error Frame	Signal detected error
Overload Frame	Request transmission delay

Messages Summary

- 4 frame types: data, remote, error, overload.
- 2 ID formats: 11-bit and 29-bit.
- Data payloads: 8 bytes (Classical), 64 bytes (FD), 2048 bytes (XL)

Data Frame

Sent for publishing info on the bus – with Data Field depending on Version.

The Node initiating the message must first compete on “who has the MIC” (ID Priority).

Remote Frame

Sent when requesting data from other node on the bus. There is no data field (shorter message).

Remote Transmission Request (RTR) = '1'.

Error Fame

Sent as 6 Consecutive bits: 000000 is Active strong Error, 111111 is Passive weak Error, followed by 8 '1's for resetting the bus (11111111). All nodes turn passive, the Node transmitted the last message will retry to re-transmit after 3Bits pause.

Overload Frame

It is identical to the Active Error Sent By a node to request more time when buffer is full. It “buys” some time for that Node and the Last message will be transmitted again.

CONSECUTIVE BITS RULE

This rule maintains synchronization, Prevents DC Component and provides edges for clock recovery.

After 5 consecutive identical bits, insert complement bit: 00000 stuffed with '1', 11111: stuffed with '0'

Example:

Original: 1 1 1 1 1 0 0 0 0 1

Stuffed: 1 1 1 1 1 [0] 0 0 0 0 [1] 1

NOTES & TIPS

Stuffed Bits (in red)
become part of the bus, hence
Counted in consecutive bits check

DATA FRAME STRUCTURE

CAN 2.0A – Standard Frame

SOF 1 bit	ID 11 bits	RTR 1 bit	IDE / r0 2 bits	DLC 4 bits	DATA 0–64 bits	CRC 15 bits	CRC DEL 1 bit	ACK + DEL 2 bits	EOF 7 bits
TOTAL BITS 44–108 bits		MAX BITRATE 1 Mbps		PAYLOAD 0–8 bytes					

CAN 2.0B – Extended Frame

SOF 1 bit	Base ID 11 bits	SRR / IDE 2 bits	Ext ID 18 bits	RTR / r1 / r0 3 bits	DLC 4 bits	DATA 0–64 bits	CRC 15 bits	CRC DEL 1 bit	ACK + DEL 2 bits	EOF 7 bits
TOTAL BITS 54–118 bits		MAX BITRATE 1 Mbps		PAYLOAD 0–8 bytes						

CAN FD – Flexible Data Rate

SOF 1 bit	ID 11 / 29 bits	FDF / r0 2 bits	BRS 1 bit	ESI 1 bit	DLC 4 bits	DATA 0–512 bits	Stuff Cnt 4 bits	CRC 17 / 21 bits	CRC DEL 1 bit	ACK + DEL 2 bits	EOF 7 bits
TOTAL BITS 46–580 bits		ARBITRATION RATE ≤ 1 Mbps		DATA RATE ≤ 8 Mbps		PAYLOAD 0–64 bytes					

SOF	Start Of Frame
ID	Identifier
RTR	Remote Transmission Request
IDE	ID Extended
r0/r1	Reserved
DLC	Data Length Code
Data	Payload
CRC	Cyclic Redundancy Check
ACK	Acknowledge
EOF	End Of Frame
FDF	FD Format
BRS	Bit Rate Switch
ESI	Error State Indicator
SRR	Substitute Remote Request
DEL	Delimiter

CAN ECO-System

CAN TRANSPORT METHODS

Method	Description	Typical Use Case
Native CAN	Direct CAN wiring	Embedded systems
USB	PC via CAN-USB	PC analysis/logging
Ethernet	CAN over TCP/UDP	Long-distance networks
RS-232/RS-485	Legacy tunneling	Industrial retrofits
WiFi	Wireless gateway	Remote monitoring

INDUSTRIAL PROTOCOLS

Protocol	Physical Layer	Speed (Kbps)	Typical Domain
CANopen	CAN 2.0A/B	1000	Industrial , medical
DeviceNet	CAN 2.0A	500	Factory automation
J1939	CAN 2.0B (29-bit)	500	Heavy vehicles, agriculture

CAN DEVICE PROFILES

Profile	Application	Key Feature
DS401 (CiA 401)	Generic I/O modules	Digital/analog I/O standardization
DS402 (CiA 402)	Drives & motion control	State machine, position/velocity/torque
DS403 (CiA 403)	HMI devices	Human-machine interface elements
DS404 (CiA 404)	Measuring & control	Closed-loop control devices
DS405 (CiA 405)	IEC 61131-3 PLCs	Programmable logic controllers
DS406 (CiA 406)	Encoders	Position feedback devices
DS407 (CiA 407)	Local transport systems	AGVs, conveyors
DS408 (CiA 408)	Hydraulic drives	Hydraulic proportional valves
DS410 (CiA 410)	Inclinometers	Angle measurement devices
DS412 (CiA 412)	Medical devices	Medical equipment profiles
DS413 (CiA 413)	Truck gateways	J1939 ↔ CANopen conversion
DS417 (CiA 417)	Lift control systems	Elevators, lifts
DS418 (CiA 418)	Battery modules	Battery management systems
DS419 (CiA 419)	Battery chargers	Charging systems

Bus Access

CSMA TOPOLOGY

Carrier Sense Multiple Access – Nodes compete for their turn on the bus.

The Full Sequence

1. Node waits for bus idle
2. Begins transmission
3. Monitors bus while transmitting
4. Compares transmitted vs received bit
5. If conflict (sent recessive, read dominant: loses arbitration)
6. Losing node becomes receiver

NOTES & TIPS

Dominant Node Validates each transmitted bit on the bus.
If conflict occurs, Dominant Node Loses its "turn" (becomes Receiver)

BUS PRIORITY

Each Node holds a Unique ID (Same IDs are not allowed on the same bus). When multiple Nodes are trying to be dominant, they are chosen by priority (lowest ID wins). The losing Nodes will retry again to get their turn.

Bits Compared

CAN2.0A	CAN2.0B	CAN FDstd	CAN FDext
12Bits	32Bits	12Bits	32Bits
ID+RTR	ID+RTR+SRR+IDE	ID+RRS	ID+RRS+SRR+IDE

Key Rules

- 12Bit (2.0A) or 32Bits (2.0 / FD) are compared sequentially until one node wins.
- Completed within identifier field
- Data frames (RTR=0) always beat Remote frames (RTR=1) at bit 11.
- Lowest numerical value wins because dominant (0) overrides recessive (1).
- No data lost during arbitration
- No bandwidth wasted on re-transmission

Example

Node A: 0x123 (001 0010 0011 0) ID + RTR=0 - Data

Node B: 0x105 (001 0000 0101 0) ID + RTR=0 - Data

Node C: 0x111 (001 0001 0001 1) ID + RTR=1 - Remote

NOTES & TIPS

If two nodes Share the same ID, When they are compared, both "Wins"! However, the validation Continues When the data is sent. The first conflict leads to error Message sent by Node A(000000), And when Node B reads the error, It immediately becomes receiver.

Node	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	RTR	Rec/Dom
Node A	0	0	1	0	0	1	0	0	0	1	1	0	Receiver
Node B	0	0	1	0	0	0	0	0	1	0	1	0	Dominant
Node C	0	0	1	0	0	0	1	0	0	1	1	1	Receiver

Winner: Lowest ID (Node B with 0x105)

Error Detection

ERROR DETECTION MECHANISMS

Mechanism	Detects	Coverage
CRC	Bit errors	Up to 5 random errors, bursts <15 bits
Bit Monitoring	Hardware faults	All transmitted bits verified
Bit Stuffing	Sync loss	6+ consecutive identical bits
Frame Check	Format errors	Fixed field violations
ACK Check	Reception failure	No acknowledgment received

ERROR TYPES

- Bit Error: Transmitted bit $\neq$ monitored bit
- Stuff Error 6+ consecutive identical bits in stuffed fields
- CRC Error: Calculated CRC $\neq$ received CRC
- Form Error: Fixed-form field contains illegal bits
- ACK Error: No dominant bit in ACK slot

ERROR FLAGS

Active Error flag is much stronger than passive

- Active Error Flag: 6 dominant bits 000000 (error-active nodes)
- Passive Error Flag: 6 recessive bits 111111 (error-passive nodes)

ERROR COUNTERS

Two Counters:

- TEC (Transmit Error Counter)
- REC (Receive Error Counter)

Counter Rules:

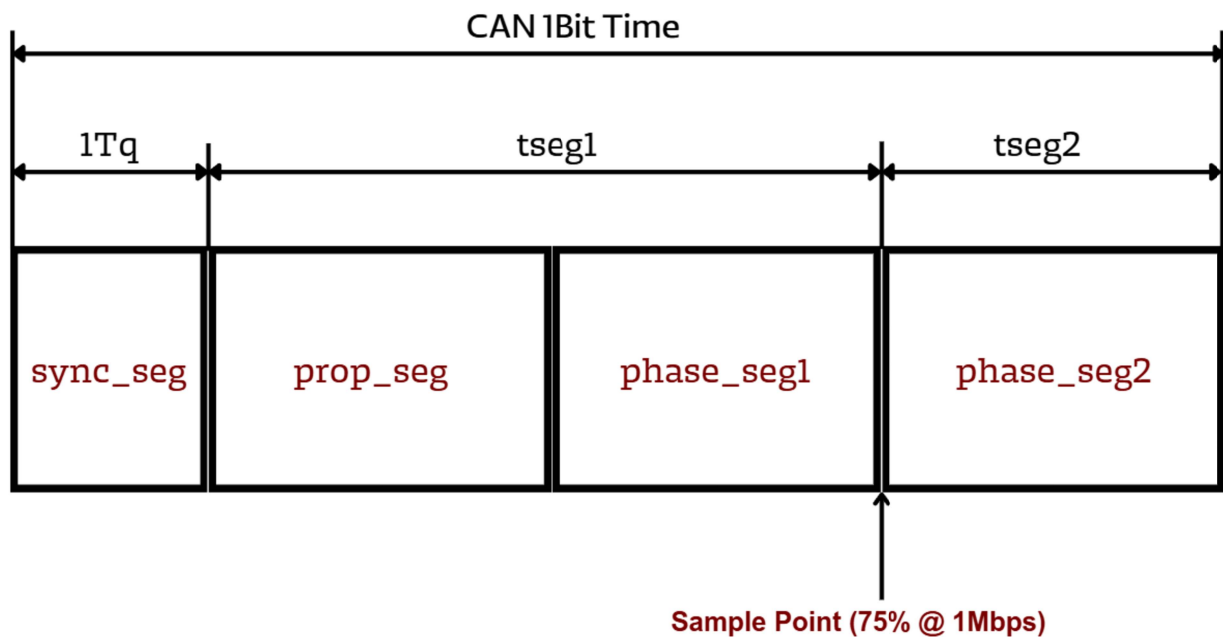
Event	Transmit Errors (TEC)	Receive Errors (REC)
Transmit error	+8	No Change
Receive error	No Change	+1
Successful TX	-1	No Change
Successful RX	No Change	-1 (if REC>0)

NOTES & TIPS

One transmit error requires
8 successful transmissions to recover
Design with margin.
Single bad message costs 8x more
than a receive error

Bit Timing

BIT TIME STRUCTURE



Time Segments

Segment	Length	Purpose
SYNC_SEG	1Tq	Synchronization, edge expected
PROP_SEG	1-8Tq	Propagation delay compensation
PHASE_SEG1	1-8Tq	Phase error compensation (lengthen)
PHASE_SEG2	2-8Tq	Phase error compensation (shorten)

SAMPLE POINT

Recommended Sample Point

Bit Rate	Sample Point
1Mbps	75-80%
500Kbps	80-87.5%
250Kbps	87.5%
125Kbps	87.5%

Sampling

- Single sample: Sample once at sample point (high speeds)
- Triple sample: Majority vote of 3 samples (noise immunity)

BIT SYNCHRONIZATION

Hard Synchronization

- Occurs at Start of Frame (SOF)
- Resets bit timing to SYNC_SEG

Re-synchronization

- Occurs on recessive-to-dominant edges
- Adjusts PHASE_SEG1 (lengthen) or PHASE_SEG2 (shorten)
- Maximum adjustment: 1-4Tq

Synchronization Rules

1. Only one synchronization per bit time
2. Edge must differ from previous sample point value
3. Hard sync on SOF during bus idle
4. Re-Sync on qualifying edges during frame

BIT CALCULATION EXAMPLE

Desired Bit Rate: 500Kbps, 16 MHz oscillator,
Desired Bit width: $1/500 \text{ kHz} = 2\mu\text{sec}$ (16Tq per bit)

$$Tq = 2\mu\text{s} / 16 = 125\text{ns}$$

$$\text{BRP} = (16 \text{ MHz} \times 125\text{ns}) - 1 = 1$$

$$\text{SYNC_SEG} = 1Tq$$

$$\text{PROP_SEG} = 3Tq \text{ (bus delay)}$$

$$\text{PHASE_SEG1} = 8Tq$$

$$\text{PHASE_SEG2} = 4Tq$$

$$\text{Sample Point} = (1+3+8)/16 = 75\% \text{ (from Bit Length)}$$

NOTES & TIPS

Target 75-87.5% sample point
for robust communication

Earlier sample points (75%) better for long buses
or slow transceivers Later sample points (87.5%)
better for short, high-speed networks

COMMON BIT RATES & SETTINGS

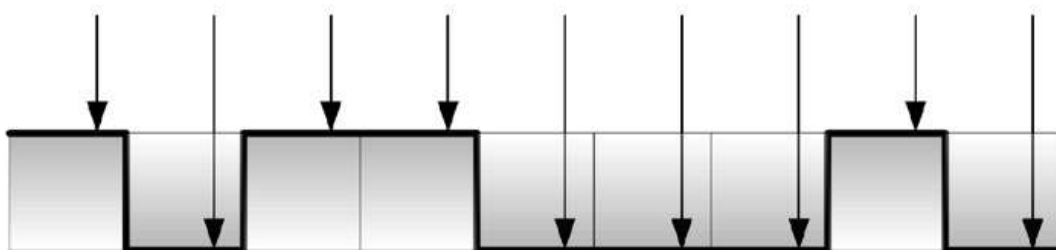
1 Mbps (40m max, 16 MHz osc):

- BRP=0, TQ=62.5ns, 16 TQ/bit
- SYNC=1, PROP=2, PH1=7, PH2=6

500 kbps (100m max, 16 MHz osc):

- BRP=1, TQ=125ns, 16 TQ/bit
- SYNC=1, PROP=2, PH1=7, PH2=6

Bit Sample Point (75%)



Hardware Implementation

PCB GUIDELINES

- Keep CAN_H and CAN_L close together
- Route as differential pair (matched length)
- Avoid routing near high-speed signals
- Use ground plane
- Place termination close to transceiver

PHYSICAL LAYER

CAN uses differential voltage between two wires:

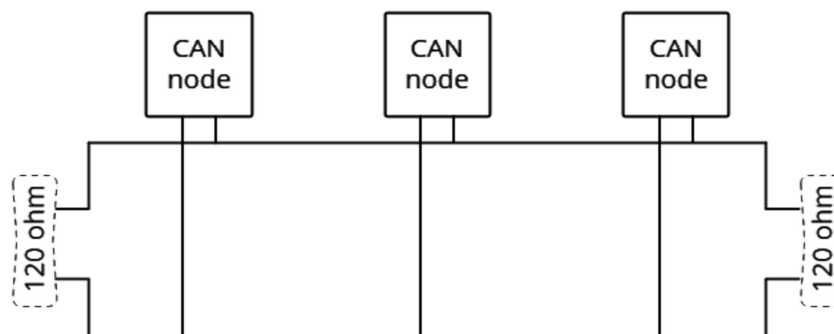
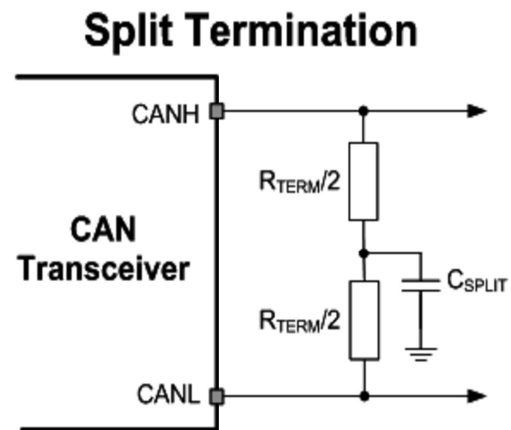
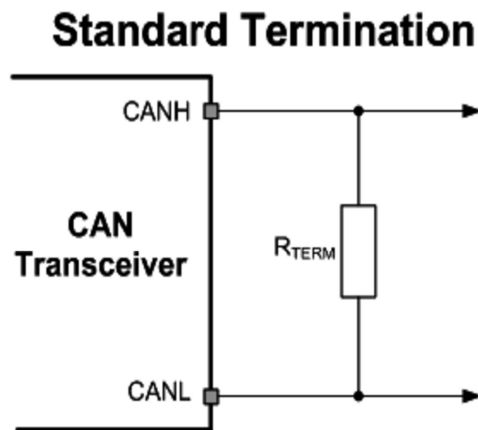
State	CAN_H	CAN_L	Differential	Logic
Dominant	3.5V	1.5V	2.0V	0
Recessive	2.5V	2.5V	0V	1

CAN BUS TERMINATION

- 120ohm at each end of the bus
- Results in 60ohm between CAN_H and CAN_L
- Prevents signal reflections
- Must be at physical ends of the linear bus topology
- Optional split termination (60ohm + 60ohm with cap to GND)

NOTES & TIPS

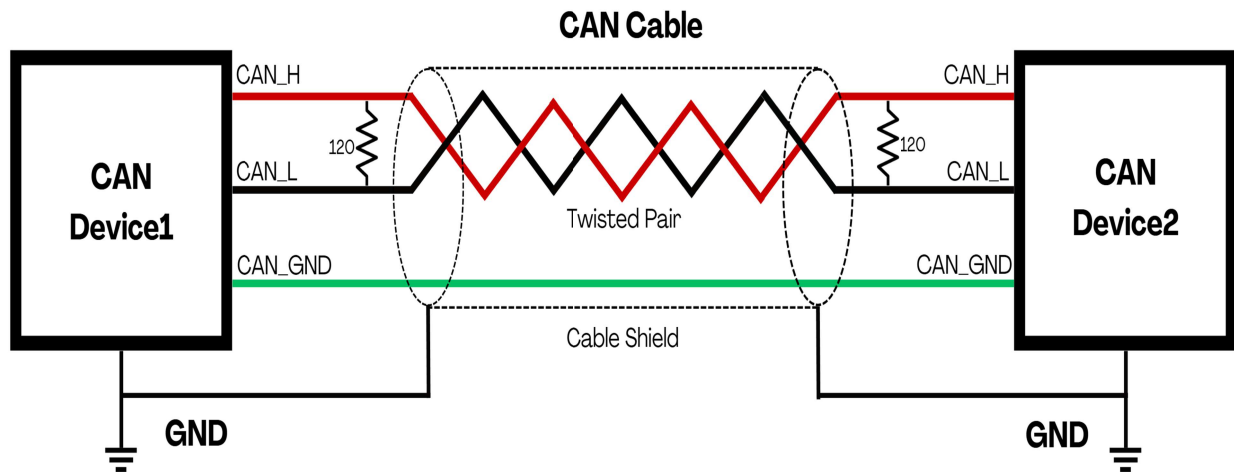
Use split termination in noisy environment.
The capacitor (47nF typical) filters high-freq EMI while maintaining proper 120Ω termination



CABLE SPECIFICATIONS

Recommended:

- Twisted pair (minimum 1 twist/inch)
- AWG 26-22 gauge wire
- Shielded cable for High-EMI environments



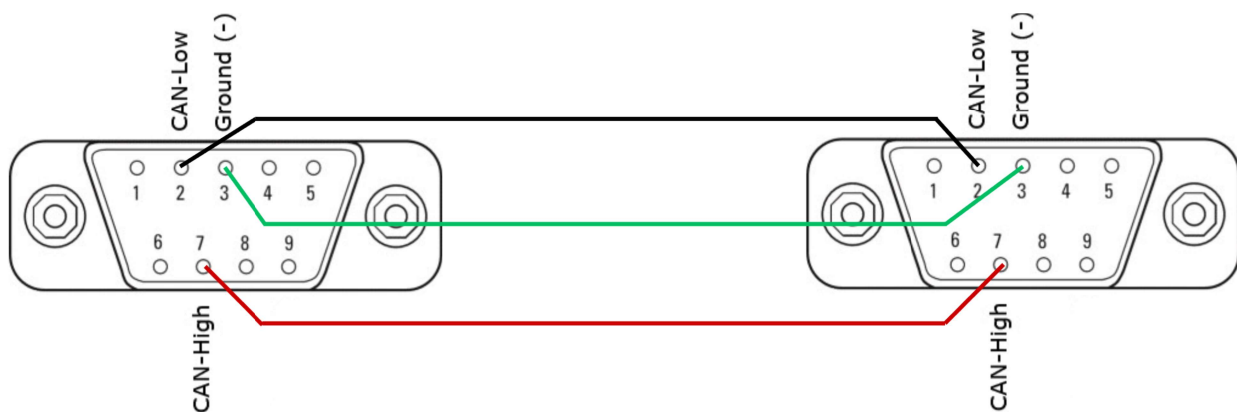
COMMON CONNECTOR (DB9)

D-SUB 9Pin:

- Only 3 Pins are used
- Shield can be Grounded as well
- Usually Device is Female
- Very Common among CAN

NOTES & TIPS

Other Signals such as CAN V+ or CAN_SHLD Are optional and can could be included on the connector as well.



TRANSCEIVER ICS

Common Integrated Circuits:

- MCP2551 (Microchip)
- TJA1050 (NXP)
- SN65HVD230 (TI)
- L9616 (ST)

Functions:

- Converts digital signals to differential voltages
- Provides short-circuit and thermal protection
- Implements slope control for EMI reduction
- Supports sleep/standby modes

NOTES & TIPS

Trans input:

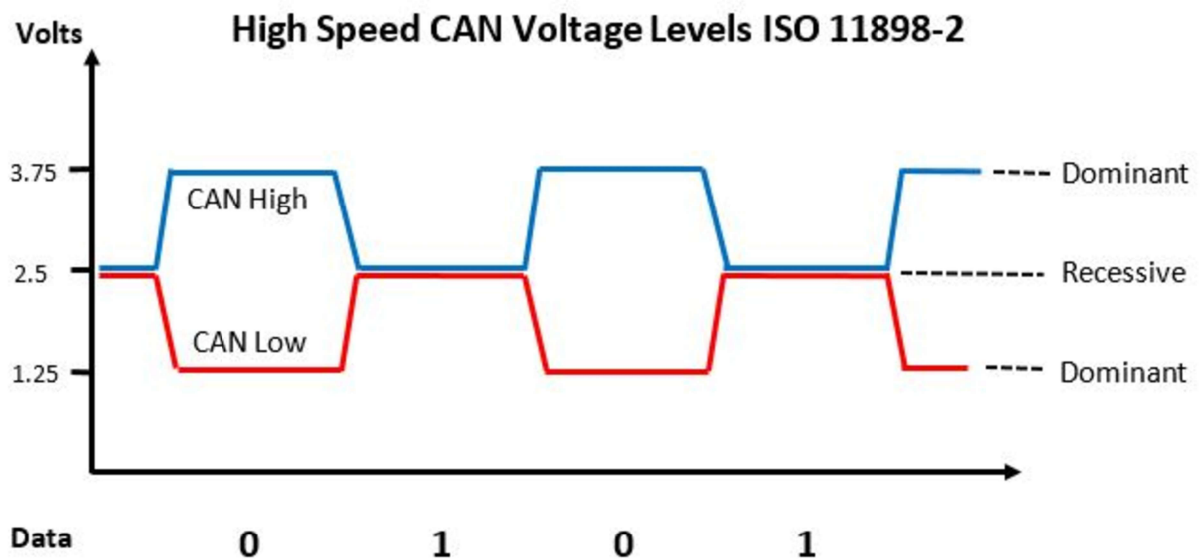
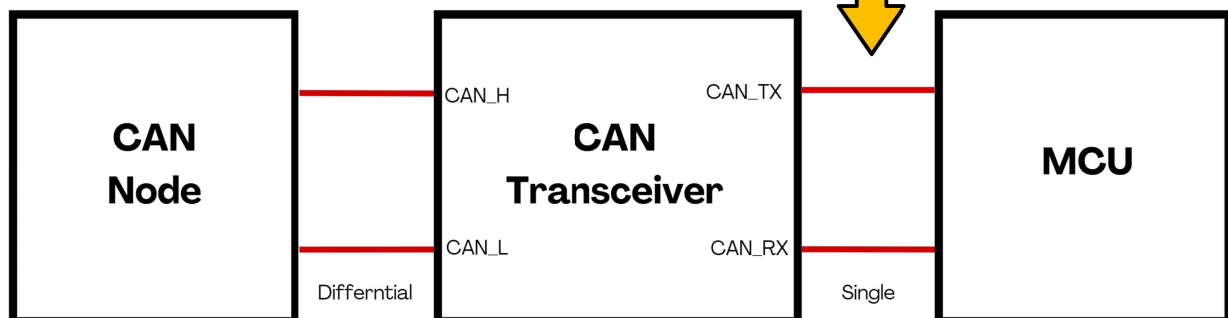
CAN_H = 2.5V/3.5V

CAN_L = 2.5V/1.5V

Trans Output:

CAN_TX = 3.3V

CAN_RX = 0V



Firmware Implementation

BASIC CAN FW (BASED ON STM32F407)

CubeMX Set-UP

- Enable CAN1 peripheral
- CAN_TX → PA12 | CAN_RX → PA11
- Enable interrupts if needed
- For 500kbps: $42\text{MHz}/(500\text{kHz} \times 7) = 12$ so Prescaler = 6
- Time Quanta: 7 (1Tq Sync + 6Tq Bit)
- Mode: Normal
- Auto Bus Off Management: Disable
- Auto Wake Up Mode: Disable
- Auto Retransmission: Enable
- Receive FIFO Locked Mode: Disable
- Transmit FIFO Priority: Disabled

Initialization

```
#include "stm32f4xx_hal.h"

CAN_HandleTypeDef hcan1;
CAN_TxHeaderTypeDef TxHeader;
CAN_RxHeaderTypeDef RxHeader;
uint8_t TxData[8];
uint8_t RxData[8];
uint32_t TxMailbox;

void CAN_Init(void) {
    hcan1.Instance = CAN1;
    hcan1.Init.Prescaler = 6;
    hcan1.Init.Mode = CAN_MODE_NORMAL;
    hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
    hcan1.Init.TimeSeg1 = CAN_BS1_6TQ;
    hcan1.Init.TimeSeg2 = CAN_BS2_1TQ;
    hcan1.Init.TimeTriggeredMode = DISABLE;
    hcan1.Init.AutoBusOff = DISABLE;
    hcan1.Init.AutoWakeUp = DISABLE;
    hcan1.Init.AutoRetransmission = ENABLE;
    hcan1.Init.ReceiveFifoLocked = DISABLE;
    hcan1.Init.TransmitFifoPriority = DISABLE;

    if (HAL_CAN_Init(&hcan1) != HAL_OK) {
        Error_Handler();
    }

    // Configure filter to accept all messages
    CAN_FilterTypeDef filter;
    filter.FilterBank = 0;
    filter.FilterMode = CAN_FILTERMODE_IDMASK;
    filter.FilterScale = CAN_FILTERSCALE_32BIT;
    filter.FilterIdHigh = 0x0000;
    filter.FilterIdLow = 0x0000;
    filter.FilterMaskIdHigh = 0x0000;
    filter.FilterMaskIdLow = 0x0000;
    filter.FilterFIFOAssignment = CAN_RX_FIFO0;
    filter.FilterActivation = ENABLE;
    filter.SlaveStartFilterBank = 14;

    HAL_CAN_ConfigFilter(&hcan1, &filter);

    // Start CAN peripheral
    HAL_CAN_Start(&hcan1);

    // Activate RX interrupt (optional)
    HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);
}
```

Transmit Function

```
uint8_t CAN_Send(uint32_t id, uint8_t* data, uint8_t length)
{
    TxHeader.StdId = id;
    TxHeader.ExtId = 0;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.DLC = length;
    TxHeader.TransmitGlobalTime = DISABLE;

    if (HAL_CAN_AddTxMessage(&hcan1, &TxHeader, data, &TxMailbox) != HAL_OK)
    {
        return 0; // Transmission failed
    }

    while(HAL_CAN_IsTxMessagePending(&hcan1, TxMailbox));
    return 1; // Success
}
```

Receive Function

```
uint8_t CAN_Receive(uint32_t* id, uint8_t* data, uint8_t* length)
{
    if (HAL_CAN_GetRxFifoFillLevel(&hcan1, CAN_RX_FIFO0) > 0)
    {
        if (HAL_CAN_GetRxMessage(&hcan1, CAN_RX_FIFO0, &RxHeader, data) == HAL_OK)
        {
            *id = RxHeader.StdId;
            *length = RxHeader.DLC;
            return 1; // Message received
        }
    }
    return 0; // No message
}
```

Interrupt Handler (Optional)

```
// In stm32f4xx_it.c
void CAN1_RX0_IRQHandler(void)
{
    HAL_CAN_IRQHandler(&hcan1);
}

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    uint8_t rxData[8];
    uint32_t rxId;

    if (hcan->Instance == CAN1)
    {
        HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, rxData);
        rxId = RxHeader.StdId;
        Process_CAN_Message(rxId, rxData, RxHeader.DLC);
    }
}
```

Complete Example: Echo Application

```
int main(void) {
    HAL_Init();
    SystemClock_Config();
    CAN_Init();

    uint8_t txData[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};
    uint8_t rxData[8];
    uint32_t rxId;
    uint8_t rxLength;

    while (1)
    {
        // Send a message every second
        if (CAN_Send(0x123, txData, 8))
        {
            // Toggle LED on successful transmission
            HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
        }

        // Check for received messages
        if (CAN_Receive(&rxId, rxData, &rxLength))
        {
            // Echo back received message with ID+1
            CAN_Send(rxId + 1, rxData, rxLength);
        }

        HAL_Delay(1000);
    }
}
```


CAN Security

CAN VULNERABILITY

Security Issues

Vulnerability	Risk Level
No Authentication	Critical
No Encryption	High
Broadcast Nature	Medium
ID Spoofing	Critical
DoS Susceptibility	Critical

Common Attack Types

Attack	Method	Impact
Bus Flooding	Spam highest priority ID (0x000)	Complete DoS
Replay Attack	Capture & retransmit commands	Unauthorized actions
Fuzzing	Random/malformed messages	ECU crash
Spoofing	Send messages with legitimate IDs	Control hijack
Diagnostic Abuse	UDS manipulation	ECU reprogramming

COUNTERMEASURES

Hardware Security

- OBD-II port lockout
- Gateway isolation between buses
- Separate critical (safety) from infotainment
- Tamper-evident seals on ECUs

Software Security

- CMAC/HMAC message authentication (64-128 bits)
- Intrusion detection (frequency/timing monitoring)
- Secure boot with signature verification
- Seed/key diagnostic authentication

SECURITY PROTOCOLS

Standard	Description
SecOC	AUTOSAR message authentication
CANcrypt	ISO 16363-2 authenticated encryption
ISO 21434	Automotive cybersecurity lifecycle

ANOMALY DETECTION:

- Message interval outside $\pm 20\%$ baseline
- Unexpected message IDs
- Error frame rate $> 10/\text{second}$
- Bus load $> 80\%$ sustained

CAN Common Issues

High Error Rates:

- Check termination (measure 60ohm between CAN_H/L)
- Verify bit timing configuration
- Check for EMI sources
- Inspect cable quality and connections

Intermittent Communication

- Check power supply stability
- Verify ground connections
- Look for loose connectors
- Check transceiver sleep modes

Bus Off Condition:

- Identify faulty node (error counters)
- Check for timing mismatches
- Verify correct bit rate across network
- Inspect for hardware issues

Message Loss:

- Reduce bus load (< 70%)
- Increase RX buffer size
- Adjust message priorities
- Implement flow control

BEST PRACTICES

Design

- Standard bit rates (125k, 250k, 500k, 1M)
- Keep bus length appropriate for bit rate
- Minimize stub lengths (<0.3m)
- Use shielded twisted pair in noisy environments

NOTES & TIPS

Standard bit rates ensure compatibility across vendors. Non-standard rates may not be supported by all devices. Stick to 125k/250k/500k/1M for maximum interoperability

Software

- Implement watchdog timers
- Handle all error states
- Log diagnostic information
- Use version control for message definitions

Testing

- Test with multiple vendors' nodes
- Verify electromagnetic compatibility
- Perform long-term reliability testing
- Validate at temperature extremes

Documentation

- Maintain network database (DBC file)
- Document message definitions
- Record bit timing parameters
- Keep hardware schematics updated

CAN Tools

DEVELOPMENT & ANALYSIS TOOLS

Tool	Platform	Purpose
SocketCAN	Linux	Native CAN
CAN-Uutils	Linux	CLI tools
Wireshark	Cross	Analysis
BUSMASTER	Windows	Sim/analysis
Candump	Linux	Send/recv
CANalyzer0r	Linux	Security
Kayak	Java	Monitoring
CANTact	Cross	Hardware

HARDWARE INTERFACES

Device	Interface	Features
CANTact	USB-CAN	Open, Python
USB2CAN	USB-CAN	Compact, SocketCAN
PEAK PCAN	USB-CAN	Pro, stable
Pi+CAN HAT	Embedded	All-in-one
ESP32+CAN	WiFi/CAN	IoT

SIMULATION AND TESTING

Tool	Type	Features
CANalyzer LE	Free	4 nodes analysis
CANopen Magic	Free	CANopen config
Kvaser CANKing	Free	Monitoring
SavvyCAN	Open	Advanced

Quick CAN Tips

- 120ohm at BOTH ends
- Measure 60ohm between CAN_H/L
- Use split termination for EMI ($2 \times 60\text{ohm} + \text{cap}$)
- Keep differential pair close ($<5\text{mm}$)
- Match trace lengths ($\pm 10\text{mm}$)
- Use ground plane under CAN traces
- Place transceiver near connector
- Separate analog/digital power
- $100\text{nF} + 10\mu\text{F}$ near transceiver
- Single-point grounding
- Check for ground loops
- Calculate voltage drop for long runs
- Use 75–80% sample point for 1 Mbps
- Calculate using formula: $T_q = 2 \times (\text{BRP} + 1) / F_{\text{osc}}$
- Account for $\pm 0.1\%$ crystal tolerance
- Synchronize settings across network
- Test bit timing with scope
- Validate sample points
- Assign low IDs (0x000–0x0FF) to critical
- Keep critical messages short
- Use priority groups
- Calculate worst-case response time
- Design the bus for $<30\%$ normal, $<70\%$ peak
- Use event-triggered instead of periodic
- Include 20% stuffing overhead
- Leave 30% capacity for growth
- Use separate buses for different speeds
- Gateway between different bit rates
- Synchronize sample points
- Size buffers for worst-case load
- Implement overflow counters
- Use non-blocking transmit
- Check TXE flag before sending
- Use DMA for zero-copy reception
- Circular buffer for high-frequency messages
- Implement duplicate detection
- Use address claiming (J1939)
- Store address in non-volatile memory
- Include manual address setting option
- Monitor TEC/REC thresholds
- Implement bus-off recovery
- Automatic retry with backoff
- Log errors with timestamps

Appendix: Quick Reference

UDS (UNIFIED DIAGNOSTIC SERVICES) - ISO 14229

Service ID	Description	Common Use
0x10	Diagnostic Session Control	Switch modes, ECU programming
0x11	ECU Reset	Reset after flashing
0x14	Clear Diagnostic Info	Clear DTCs
0x19	Read DTC Information	Read fault codes
0x22	Read Data By Identifier	Read sensor values
0x2E	Write Data By Identifier	Configuration
0x31	Routine Control	Self-tests
0x34	Request Download	Flash programming
0x36	Transfer Data	Data transfer
0x37	Request Transfer Exit	Complete programming

OBD-II POPULAR PARAMETER

PID	Description	Units	Formula
0x0C	Engine RPM	RPM	$(A \times 256 + B) \div 4$
0x0D	Vehicle Speed	km/h	A
0x05	Engine Coolant Temp	°C	A - 40
0x0F	Intake Air Temp	°C	A - 40
0x11	Throttle Position	%	$(A \times 100) \div 255$
0x10	MAF Air Flow Rate	g/s	$(256 \times A + B) \div 100$
0x04	Load	%	$A \times 100 \div 255$
0x0B	Manifold Pressure	kPa	A
0x21	O2 Sensor 1 (Bank 1)	V	$A \div 200 - 0.1$
0x06	Fuel System Status	-	Binary status